

Midterm Report:
RTL Design and Simulation Verification of a
Streaming Denoise Accelerator

Yicheng Wang
Student ID: 123090586

April 13, 2026

Abstract

This report presents the midterm progress of the continued hardware–software co-design project for accelerating an Image Signal Processing (ISP) pipeline on the BeagleV-Fire RISC-V + FPGA platform. Previous work identified the denoise stage as the main computational bottleneck in the software ISP pipeline. Based on that result, this stage focuses on the RTL design and simulation verification of a streaming denoise accelerator.

The implemented hardware module processes 8-bit grayscale pixel streams and applies a fixed 3×3 Gaussian-like denoise kernel. The core uses line buffers and a sliding-window structure to form the required pixel neighborhood. It was first verified using a small 8×8 matrix test, and then tested with a real-image simulation flow where a 64×64 grayscale image was converted into hexadecimal input data. A Python reference model was used as an independent golden model, and the Verilog output matched the expected result at pixel level.

The raw denoise core was then wrapped with AXI4-Stream input and output interfaces. A basic AXI4-Stream test passed under normal streaming conditions. A later backpressure test exposed an output data-loss problem when the downstream interface was not ready. This issue was fixed by adding an output FIFO. Finally, a controlled wrapper was implemented with `start`, `busy`, and `done` signals. The controlled wrapper successfully processed one frame and generated 3844 output pixels for a 64×64 input image. This completes the RTL-level verification stage and prepares the design for later Libero implementation and official BeagleV-Fire gateware integration.

Contents

1	Introduction	4
2	Review of Previous Work	5
3	RTL Design of the Denoise Core	5
3.1	Design Objective	5
3.2	Gaussian-Like Kernel	6
3.3	Line Buffer and Sliding Window	6
3.4	Valid Output Region	6
4	Functional Simulation and Verification	7
4.1	Basic 8×8 Test	7
4.2	Waveform Inspection	8
4.3	Real Image Hex-Based Simulation	8
4.4	Visual Result	9
5	AXI4-Stream Wrapper Design	10
5.1	Motivation	10
5.2	Interface Mapping	10
5.3	Basic AXI4-Stream Test	11
6	Backpressure Handling and FIFO Fix	12
6.1	Problem	12
6.2	FIFO Fix	12
6.3	Backpressure Test Result	12
7	Controlled AXI-Stream Wrapper	13
7.1	Frame-Level Control	13
7.2	Control Behavior	14
7.3	Controlled Wrapper Test	14
8	Conclusion and Next Steps	15

1 Introduction

Image Signal Processing (ISP) pipelines are common in embedded vision systems. A typical ISP pipeline may include RAW image generation or acquisition, demosaicing, auto white balance, denoising, and output image generation. Among these stages, denoising is often computationally expensive because it repeatedly applies neighborhood operations to every pixel.

In the previous stage of this project, a software-only ISP pipeline was implemented and profiled. The pipeline followed the sequence:

RGB input $\rightarrow$ fake RAW/Bayer simulation $\rightarrow$ demosaic $\rightarrow$ AWB $\rightarrow$ denoise $\rightarrow$ output

The profiling results showed that the denoise stage consumed the largest part of the execution time. Therefore, denoise was selected as the target module for FPGA acceleration.

The long-term goal of the project is to build a complete hardware–software co-design system on the BeagleV-Fire platform. In the final system, the RISC-V processor will handle software control and image I/O, while the FPGA fabric will accelerate the denoise computation. However, before integrating the design into the full BeagleV-Fire gateway, the RTL module itself must first be designed, simulated, and verified.

This midterm report focuses on the RTL-level work. The main tasks completed in this phase are:

- implementation of a streaming 3×3 denoise core;
- verification using an 8×8 matrix test;
- file-based simulation using a real grayscale image;
- comparison against a Python golden reference;
- AXI4-Stream wrapper design;
- backpressure testing and FIFO-based correction;
- controlled wrapper design with `start`, `busy`, and `done` signals.

The rest of this report is organized as follows. Section 2 reviews the project foundation. Section 3 describes the denoise core design. Section 4 presents the simulation and verification results. Section 5 explains the AXI4-Stream wrapper. Section 6 discusses the backpressure issue and FIFO fix. Section 7 presents the controlled wrapper. Section 8 concludes the report and outlines the next steps.

2 Review of Previous Work

The previous work established the motivation for this hardware accelerator. A C++ software baseline was developed for the ISP pipeline, including demosaic, auto white balance, and denoise. The denoise stage applied a fixed 3×3 Gaussian-like filter to each pixel channel.

The software profiling showed that denoise was the main runtime bottleneck. This is reasonable because a spatial denoise filter must read neighboring pixels, multiply them by kernel weights, accumulate the results, and normalize the final value. This pattern is repeated for nearly every pixel in the image.

From a hardware perspective, this operation is suitable for FPGA acceleration for several reasons:

- the computation pattern is regular;
- the kernel size is fixed;
- the same operation is repeated across the image;
- line buffers can reduce repeated memory access;
- the computation can be expressed as a streaming pipeline.

Based on this observation, the current stage moves from software profiling to RTL implementation. The goal is not yet to build the full board-level system, but to produce a correct, synthesizable, and interface-ready denoise accelerator module.

3 RTL Design of the Denoise Core

3.1 Design Objective

The first hardware module implemented in this stage is:

```
1 denoise_core_simple.sv
```

This module processes an 8-bit grayscale pixel stream. To simplify the first hardware implementation, RGB channels are not processed together. Instead, the module operates on one channel at a time. This is acceptable because the software denoise operation treats the R, G, and B channels independently.

The input pixels arrive in raster-scan order, from left to right and top to bottom. For each valid 3×3 window, the hardware computes one filtered output pixel.

3.2 Gaussian-Like Kernel

The implemented denoise filter uses the following 3×3 Gaussian-like kernel:

$$K = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

The sum of the kernel coefficients is 16. Therefore, the output pixel is calculated as:

$$P_{out} = \frac{1P_{00} + 2P_{01} + 1P_{02} + 2P_{10} + 4P_{11} + 2P_{12} + 1P_{20} + 2P_{21} + 1P_{22}}{16}$$

Since the divisor is 16, the division can be implemented as a right shift by four bits. This avoids the need for a hardware divider.

3.3 Line Buffer and Sliding Window

A 3×3 filter requires pixels from three adjacent rows. Since the input is a pixel stream, the hardware cannot directly access the whole image at once. The design therefore uses two line buffers to store previous rows and local registers to build the current 3×3 window.

The conceptual window is:

Listing 1: Conceptual 3x3 window

1	p00	p01	p02
2	p10	p11	p12
3	p20	p21	p22

Each new pixel shifts into the window as the stream advances. After the first two rows and first two columns have been received, the window becomes valid and the module begins to produce output pixels.

3.4 Valid Output Region

The current design outputs only the valid internal region of the image. It does not generate boundary pixels. For an input image of width W and height H , the output size is:

$$(W - 2) \times (H - 2)$$

For the 64×64 test image used later in this report:

$$(64 - 2) \times (64 - 2) = 62 \times 62 = 3844$$

Thus, a correct one-frame test should produce 3844 valid output pixels.

4 Functional Simulation and Verification

4.1 Basic 8×8 Test

The first testbench used a small 8×8 input matrix. The testbench generated the input values internally, sent them into `denoise_core_simple`, calculated the expected result, and compared each RTL output value with the expected value.

For an 8×8 image, the valid convolution region is 6×6 . Figure 1 shows the simulation result. Each output value is marked with [OK], and the test ends with TEST PASSED.

```
PS E:\DDA2081\fpga_isp_denoise> vvp sim.vvp
[OK] output[0][0] = 11
[OK] output[0][1] = 12
[OK] output[0][2] = 13
[OK] output[0][3] = 14
[OK] output[0][4] = 15
[OK] output[0][5] = 16
[OK] output[1][0] = 21
[OK] output[1][1] = 22
[OK] output[1][2] = 23
[OK] output[1][3] = 24
[OK] output[1][4] = 25
[OK] output[1][5] = 26
[OK] output[2][0] = 31
[OK] output[2][1] = 32
[OK] output[2][2] = 33
[OK] output[2][3] = 34
[OK] output[2][4] = 35
[OK] output[2][5] = 36
[OK] output[3][0] = 41
[OK] output[3][1] = 42
[OK] output[3][2] = 43
[OK] output[3][3] = 44
[OK] output[3][4] = 45
[OK] output[3][5] = 46
[OK] output[4][0] = 51
[OK] output[4][1] = 52
[OK] output[4][2] = 53
[OK] output[4][3] = 54
[OK] output[4][4] = 55
[OK] output[4][5] = 56
[OK] output[5][0] = 61
[OK] output[5][1] = 62
[OK] output[5][2] = 63
[OK] output[5][3] = 64
[OK] output[5][4] = 65
[OK] output[5][5] = 66
=====
TEST PASSED
=====
tb_denoise_core_simple.sv:137: $finish called at 900000 (1ps)
PS E:\DDA2081\fpga_isp_denoise> |
```

Figure 1: Basic 8×8 simulation result of `denoise_core_simple`.

This confirms that the line-buffer and sliding-window logic produces correct numerical

results for a controlled small input.

4.2 Waveform Inspection

The simulation waveform was also inspected using GTKWave. Figure 2 shows the key signals, including clock, input valid, input pixel, output pixel, output valid, and internal counters.

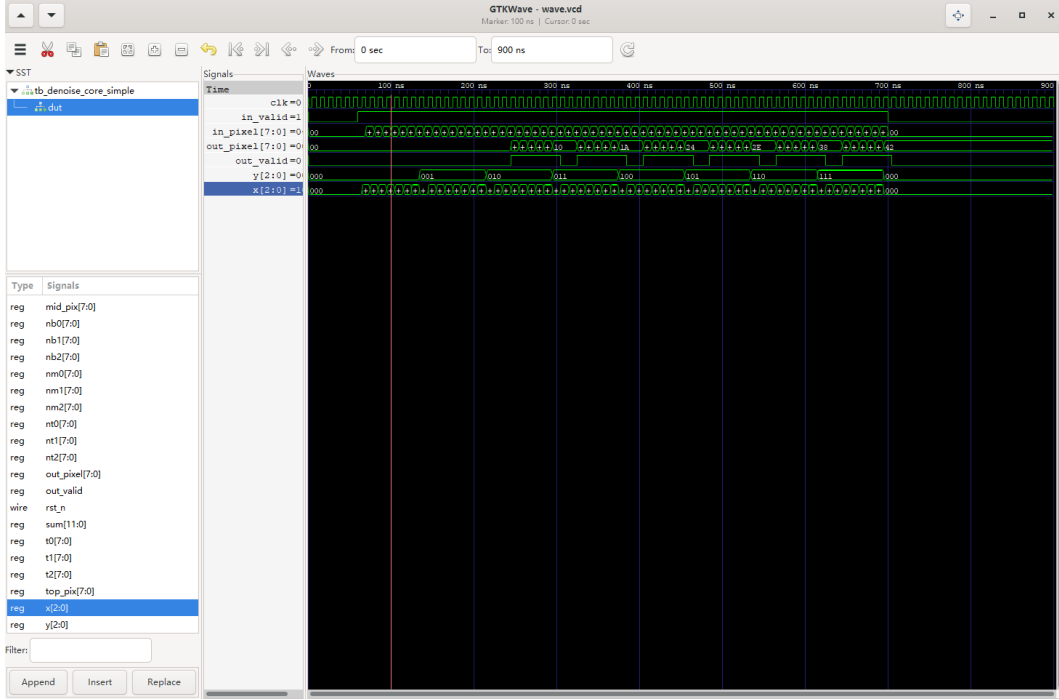


Figure 2: GTKWave waveform of the denoise core simulation.

The waveform confirms that output pixels are generated only after the 3×3 window becomes valid. It also shows that the input pixels are processed in raster-scan order.

4.3 Real Image Hex-Based Simulation

After the small test passed, the simulation was extended to use real image data. A Python script converted a real image into a 64×64 grayscale image and exported the pixel values into:

```
1 input_image.hex
```

The Verilog testbench used `$readmemh` to read the input file and stream the pixels into the denoise core. The hardware output was written to:

```
1 output_axis_hw.hex
```

A Python reference model generated:

```
1 expected_py.hex
```

Then the Python output and Verilog output were compared pixel by pixel. Figure 3 shows that the comparison passed.

```
PS E:\DDA2081\fpga_isp_denoise> py compare_denoise.py
=====
PYTHON VS VERILOG CHECK PASSED
=====
Generated: expected_py.hex
```

Figure 3: Pixel-level comparison between Python reference output and Verilog RTL output.

This result is important because the Python model is independent of the Verilog testbench. Passing this comparison gives stronger confidence that the RTL module implements the intended denoise algorithm.

4.4 Visual Result

The RTL output was also converted back into an image. Figure 4 compares the input grayscale image with the Verilog denoise output.

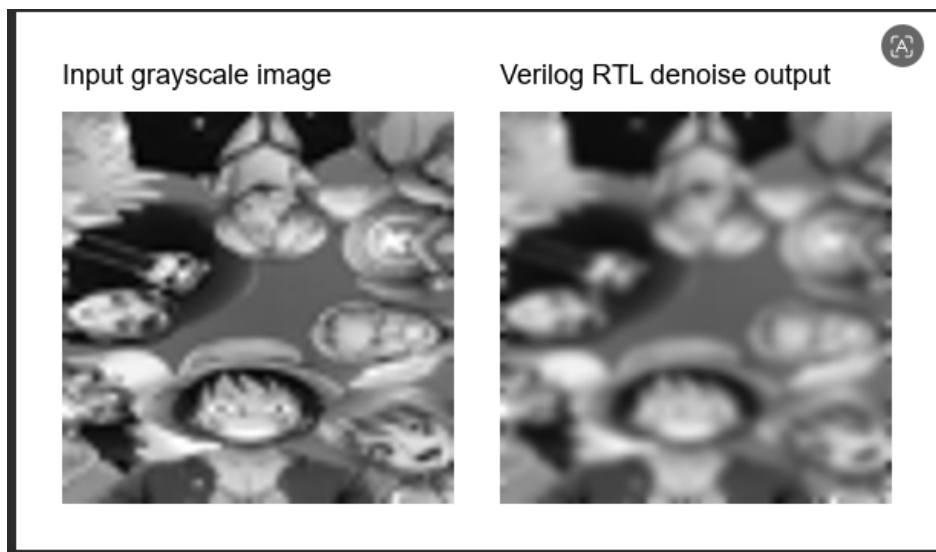


Figure 4: Visual comparison between the input grayscale image and Verilog RTL denoise output.

The output image is smoother and slightly blurred. This is expected because the implemented 3×3 Gaussian-like filter reduces high-frequency image details. The purpose of this stage is not to design a complex adaptive denoise algorithm, but to verify that a fixed convolution-based filter can be implemented correctly in RTL.

5 AXI4-Stream Wrapper Design

5.1 Motivation

The raw denoise core uses simple input-valid and output-valid signals. This is enough for early simulation, but it is not enough for later system integration. In a real FPGA system, the accelerator should be able to connect to standard streaming components such as DMA engines or other processing blocks.

Therefore, the denoise core was wrapped with an AXI4-Stream-style interface in:

```
1 denoise_axis_stream_simple.sv
```

The input interface is:

```
1 s_axis_tdata[7:0]
2 s_axis_tvalid
3 s_axis_tready
```

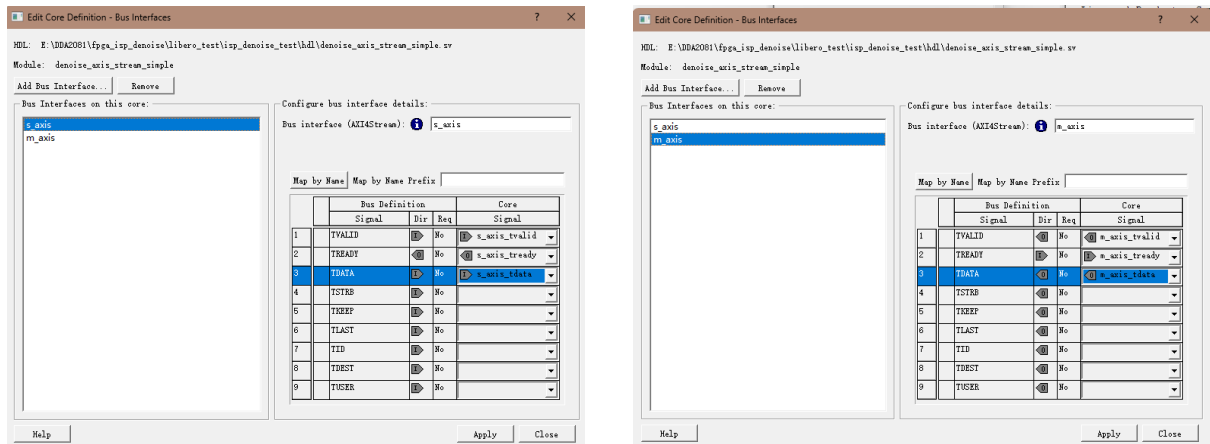
The output interface is:

```
1 m_axis_tdata[7:0]
2 m_axis_tvalid
3 m_axis_tready
```

A transfer occurs only when both `tvalid` and `tready` are high. This ready/valid handshake allows the hardware to support flow control.

5.2 Interface Mapping

The AXI4-Stream interfaces were also defined in Libero. Figure 5 shows the mapping of both the slave input and master output stream interfaces.



(a) Slave input interface

(b) Master output interface

Figure 5: AXI4-Stream bus interface mapping in Libero.

The input side maps `s_axis_tvalid`, `s_axis_tready`, and `s_axis_tdata`. The output side maps `m_axis_tvalid`, `m_axis_tready`, and `m_axis_tdata`. This prepares the module for later integration as a standard streaming component.

5.3 Basic AXI4-Stream Test

A testbench was written to verify the wrapper under normal streaming conditions. In this test, the downstream side was always ready, so there was no output stall. Figure 6 shows the simulation result.

```
PS E:\DDA2081\fpga_isp_denoise> vvp sim_axis.vvp
VCD info: dumpfile wave_axis.vcd opened for output.
[OK] AXIS output[0][0] = 11
[OK] AXIS output[0][1] = 12
[OK] AXIS output[0][2] = 13
[OK] AXIS output[0][3] = 14
[OK] AXIS output[0][4] = 15
[OK] AXIS output[0][5] = 16
[OK] AXIS output[1][0] = 21
[OK] AXIS output[1][1] = 22
[OK] AXIS output[1][2] = 23
[OK] AXIS output[1][3] = 24
[OK] AXIS output[1][4] = 25
[OK] AXIS output[1][5] = 26
[OK] AXIS output[2][0] = 31
[OK] AXIS output[2][1] = 32
[OK] AXIS output[2][2] = 33
[OK] AXIS output[2][3] = 34
[OK] AXIS output[2][4] = 35
[OK] AXIS output[2][5] = 36
[OK] AXIS output[3][0] = 41
[OK] AXIS output[3][1] = 42
[OK] AXIS output[3][2] = 43
[OK] AXIS output[3][3] = 44
[OK] AXIS output[3][4] = 45
[OK] AXIS output[3][5] = 46
[OK] AXIS output[4][0] = 51
[OK] AXIS output[4][1] = 52
[OK] AXIS output[4][2] = 53
[OK] AXIS output[4][3] = 54
[OK] AXIS output[4][4] = 55
[OK] AXIS output[4][5] = 56
[OK] AXIS output[5][0] = 61
[OK] AXIS output[5][1] = 62
[OK] AXIS output[5][2] = 63
[OK] AXIS output[5][3] = 64
[OK] AXIS output[5][4] = 65
[OK] AXIS output[5][5] = 66
=====
AXI4-STREAM WRAPPER TEST PASSED
=====
tb_denoise_axis_stream_simple.sv:163: $finish called at 900000 (1ps)
```

Figure 6: Simulation result of the basic AXI4-Stream wrapper test.

The output values match the expected result, and the terminal reports **AXI4-STREAM WRAPPER TEST PASSED**. This verifies that the wrapper can correctly connect the input stream, denoise core, and output stream under ideal conditions.

6 Backpressure Handling and FIFO Fix

6.1 Problem

Although the basic AXI4-Stream wrapper passed the normal test, it still needed to be checked under backpressure. In a real system, the downstream module may temporarily deassert `m_axis_tready`. If the accelerator produces a valid output pixel during this time and does not store it, the output data can be lost.

The first version of the wrapper had this problem. It worked when the receiver was always ready, but it could lose output pixels when `m_axis_tready` was low.

6.2 FIFO Fix

To solve this issue, a small FIFO was added between the denoise core and the AXI4-Stream master output interface:

Denoise core output → Output FIFO → AXI4-Stream output

If the denoise core produces a pixel while the downstream side is not ready, the FIFO stores the pixel. When `m_axis_tready` becomes high again, the stored pixel can be transmitted. This prevents output loss and makes the wrapper more robust.

6.3 Backpressure Test Result

After adding the FIFO, the backpressure test passed. Figure 7 shows the result.

```

PS E:\DDA2081\fpga_isp_denoise> vvp sim_axis_bp.vvp
VCD info: dumpfile wave_axis_backpressure.vcd opened for output.
[OK] BACKPRESSURE output[0][0] = 11
[OK] BACKPRESSURE output[0][1] = 12
[OK] BACKPRESSURE output[0][2] = 13
[OK] BACKPRESSURE output[0][3] = 14
[OK] BACKPRESSURE output[0][4] = 15
[OK] BACKPRESSURE output[0][5] = 16
[OK] BACKPRESSURE output[1][0] = 21
[OK] BACKPRESSURE output[1][1] = 22
[OK] BACKPRESSURE output[1][2] = 23
[OK] BACKPRESSURE output[1][3] = 24
[OK] BACKPRESSURE output[1][4] = 25
[OK] BACKPRESSURE output[1][5] = 26
[OK] BACKPRESSURE output[2][0] = 31
[OK] BACKPRESSURE output[2][1] = 32
[OK] BACKPRESSURE output[2][2] = 33
[OK] BACKPRESSURE output[2][3] = 34
[OK] BACKPRESSURE output[2][4] = 35
[OK] BACKPRESSURE output[2][5] = 36
[OK] BACKPRESSURE output[3][0] = 41
[OK] BACKPRESSURE output[3][1] = 42
[OK] BACKPRESSURE output[3][2] = 43
[OK] BACKPRESSURE output[3][3] = 44
[OK] BACKPRESSURE output[3][4] = 45
[OK] BACKPRESSURE output[3][5] = 46
[OK] BACKPRESSURE output[4][0] = 51
[OK] BACKPRESSURE output[4][1] = 52
[OK] BACKPRESSURE output[4][2] = 53
[OK] BACKPRESSURE output[4][3] = 54
[OK] BACKPRESSURE output[4][4] = 55
[OK] BACKPRESSURE output[4][5] = 56
[OK] BACKPRESSURE output[5][0] = 61
[OK] BACKPRESSURE output[5][1] = 62
[OK] BACKPRESSURE output[5][2] = 63
[OK] BACKPRESSURE output[5][3] = 64
[OK] BACKPRESSURE output[5][4] = 65
[OK] BACKPRESSURE output[5][5] = 66
=====
AXI4-STREAM BACKPRESSURE TEST PASSED
=====
tb_denoise_axis_backpressure_simple.sv:183: $finish called at 1500000 (1ps)
PS E:\DDA2081\fpga_isp_denoise> cmd /c fc expected_py.hex output_axis_backpressure_hw.hex
正在比较文件 expected_py.hex 和 OUTPUT_AXIS_BACKPRESSURE_HW.HEX
FC: 找不到差异

```

Figure 7: AXI4-Stream backpressure test result after adding the output FIFO.

The output file was also compared with the expected result, and no difference was found. This confirms that the wrapper can handle downstream stalls without losing data.

This step was an important engineering correction. The computation core was already mathematically correct, but the first stream wrapper was not fully safe under realistic flow-control conditions. The FIFO fix improved the design from a simple functional prototype into a more reliable streaming accelerator block.

7 Controlled AXI-Stream Wrapper

7.1 Frame-Level Control

The AXI4-Stream wrapper supports pixel movement, but a hardware accelerator also needs frame-level control. The processor should be able to start the accelerator, check whether it is busy, and detect when the frame has finished.

For this reason, a controlled wrapper was implemented:

```
1 denoise_controlled_axis.sv
```

This module wraps the AXI4-Stream denoise pipeline and adds:

```
1 start
2 busy
3 done
```

The signal meanings are:

- **start**: begins processing one frame;
- **busy**: indicates that the accelerator is processing;
- **done**: indicates that the frame output is complete.

7.2 Control Behavior

The wrapper follows a simple control flow:

$$\text{IDLE} \rightarrow \text{RUN} \rightarrow \text{DONE}$$

When **start** is asserted, the module enters the RUN state and begins accepting input pixels. During processing, **busy** is high. The wrapper counts valid output pixels. For a 64×64 input frame, the expected number of output pixels is:

$$62 \times 62 = 3844$$

When 3844 output pixels have been produced, **busy** becomes low and **done** becomes high.

7.3 Controlled Wrapper Test

The controlled wrapper was verified using `tb.denoise_controlled_axis.sv`. Figure 8 shows the simulation result.

```

PS E:\DDA2081\fpga_isp_denoise> iverilog -g2012 -o sim_controlled.vvp denoise_core_simple.sv denoise_axis_stream_simple.sv
sv denoise_controlled_axis.sv tb_denoise_controlled_axis.sv
tb_denoise_controlled_axis.sv:77: warning: System task ($display) cannot be synthesized in an always_ff process.
tb_denoise_controlled_axis.sv:82: warning: System task ($display) cannot be synthesized in an always_ff process.
tb_denoise_controlled_axis.sv:87: warning: System task ($fdisplay) cannot be synthesized in an always_ff process.
PS E:\DDA2081\fpga_isp_denoise> vvp sim_controlled.vvp
VCD info: dumpfile wave_controlled_axis.vcd opened for output.
[OK] CONTROLLED output[0][0] = 160
[OK] CONTROLLED output[0][1] = 159
[OK] CONTROLLED output[0][2] = 161
[OK] CONTROLLED output[0][3] = 164
[OK] CONTROLLED output[0][4] = 162
[OK] CONTROLLED output[0][5] = 161
[OK] CONTROLLED output[0][6] = 170
[OK] CONTROLLED output[0][7] = 181
[OK] CONTROLLED output[0][8] = 160
[OK] CONTROLLED output[0][9] = 92
[OK] CONTROLLED output[0][10] = 32
[OK] CONTROLLED output[0][11] = 15
[OK] CONTROLLED output[0][12] = 15
[OK] CONTROLLED output[0][13] = 14
[OK] CONTROLLED output[0][14] = 14
[OK] CONTROLLED output[0][15] = 14
[OK] CONTROLLED output[0][16] = 14
[OK] CONTROLLED output[0][17] = 14
[OK] CONTROLLED output[0][18] = 14
[OK] CONTROLLED output[0][19] = 20
=====
CONTROLLED AXIS TEST PASSED
Output count = 3844
busy=0 done=1
=====
tb_denoise_controlled_axis.sv:187: $finish called at 41150000 (1ps)
PS E:\DDA2081\fpga_isp_denoise> cmd /c fc expected_py.hex output_controlled_hw.hex
正在比较文件 expected_py.hex 和 OUTPUT_CONTROLLED_HW.HEX
FC: 找不到差异

```

Figure 8: Simulation result of the controlled AXI-Stream wrapper.

The test reports:

```

1 CONTROLLED AXIS TEST PASSED
2 Output count = 3844
3 busy = 0 done = 1

```

The generated hardware output was also compared with the Python expected output, and no difference was found. This confirms that the controlled wrapper preserves the correctness of the denoise output while adding frame-level control.

The controlled wrapper is an important intermediate step between a raw streaming filter and a software-controlled accelerator. In the future BeagleV-Fire system, the **start**, **busy**, and **done** signals can be connected to memory-mapped registers so that the RISC-V CPU can trigger and monitor the hardware module.

8 Conclusion and Next Steps

This midterm stage completed the RTL-level design and simulation verification of the denoise accelerator. The basic denoise core was implemented using line buffers and a 3×3 sliding-window structure. It was first verified with an 8×8 test matrix, and then tested using a real-image file-based simulation flow. The Verilog output matched the Python golden model at pixel level.

The design was then wrapped with AXI4-Stream interfaces. A basic AXI4-Stream test passed, but a backpressure test revealed that output data could be lost when the downstream interface was not ready. This issue was fixed by adding an output FIFO.

After this correction, the backpressure test passed and the output matched the expected result.

Finally, a controlled AXI-Stream wrapper was implemented. It added **start**, **busy**, and **done** signals and successfully processed one 64×64 input frame, producing 3844 valid output pixels. This shows that the accelerator is no longer only a raw computation block, but a controllable streaming hardware module.

The next phase will move from simulation to FPGA implementation and system integration. The verified RTL modules will be imported into Libero SoC for synthesis, place-and-route, resource analysis, and timing evaluation. After standalone FPGA validation, the design will be integrated into the official BeagleV-Fire gateway. The first safe integration path will use the CAPE/APB interface for register-level testing before attempting full AXI4-Stream or DMA-based image transfer.